Desarrollo de aplicación para asistir en el aprendizaje de las matemáticas
Development of an application to assist in learning mathematics

Trabajo de Fin de Grado
Curso 2025–2026

Autor
Jorge Polo Peyres

Director
Gonzalo Méndez Pozo
Pablo Gervás Gómez-Navarro

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

Desarrollo de aplicación para asistir en el aprendizaje de las matemáticas
Development of an application to assist in learning mathematics

Trabajo de Fin de Grado en Ingeniería Informática

Autor
Jorge Polo Peyres

Director
Gonzalo Méndez Pozo
Pablo Gervás Gómez-Navarro

Colaborador
Colaborador no definido. Usa `\colaboradorPortada`

Convocatoria: *Febrero/Junio/Septiembre 2026*

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

DIA de MES de AÑO

Dedicatoria

A Pedro Pablo y Marco Antonio, por crear
TeXiS e iluminar nuestro camino

Agradecimientos

A Guillermo, por el tiempo empleado en hacer estas plantillas. A Adrián, Enrique y Nacho, por sus comentarios para mejorar lo que hicimos. Y a Narciso, a quien no le ha hecho falta el Anillo Único para coordinarnos a todos.

Abstract

Development of an application to assist in learning mathematics

Many students struggle to fully understand all the concepts of math learned in school. One way of supporting the education in this area is to reinforce the students with problems, and if being careful when designing how to deliver the exercises and providing hints or steps towards the solution, you can achieve a very productive tool for improving the user's education.

For this project, we want to develop an application that solves this problem without falling short of personalization for the user nor lack of problems similar to the ones seen in class or with enough skill required that will induce to the resolution of simpler ones. In order to achieve this, we will address the three main issues while implementing this kind of platform:

1. How do students interact with a platform of this type, what are the problems that they often encounter while using it, and how can we build a good user interface appropriate for the user the studies made.

2. Where do we find the math problems, how do we build the database, and how do we make the problems from different sources fit the database.

3. How can we personalize the user learning experience and how would we build a recommendation system.

Keywords

Learning mathematics, Transforming datasets of problems, UI desing, MVVM pattern, .NET MAUI, Recommendation system, DKT.

Índice

Índice de figuras

Índice de tablas

Capítulo 1

Introduction

"Frase célebre dicha por alguien inteligente"
— Autor

1.1. Motivation

In the experience of many students, just what they teach in class may fall short of the expectations of the students, or on the other hand, the lessons may be too demanding for them. Regardless of which of these situations may be, the most effective solution is to provide some support to the student from a math tutor. However, this is not the most affordable option and with all the recent advances in technology and Artificial Intelligence, there has to be an effective way to substituting the math tutor with a software application.

As a matter of fact there has been plenty of programs that come to solve this issue (REFERENCIA) and we will be studying their implementation. Nevertheless, I have not been successful in finding one that achieves the results in teaching math the way the official education system gives, or at least fully supporting it on its own.

1.2. Objectives

Develop an application that simulates a math tutor by providing a large library of math problems that are labeled by topic, course and difficulty. Build a well designed UI, so that the use of the application is engaging for the students. Finally, design a recommendation system that uses the interests and interactions of one student to accurately show him the top-k problems that he should do to maximize his learning results.

1.3. Work plan

1.3.1. Application Layer: User Interface & Communication

Tabla 1.1: Implementation and Alternatives for the Application Layer

Component	Chosen Implementation	Alternative Implementations
Frontend UI Framework	**.NET MAUI** (C#, XAML) with **MVVM** pattern. Unified single codebase for native mobile/desktop apps.	**Flutter** (Dart): Excellent for highly animated UIs and fast Hot Reload. **React Native** (JavaScript/TypeScript): Large community adoption and ecosystem. **Kotlin Multiplatform** (KMP): Shares business logic, but UI requires platform-specific layers.
Mathematical Rendering	**WebView with MathJax/KaTeX:** Renders LaTeX strings to HTML for high-quality, reliable cross-platform display.	**GraphicsView + CSharpMath/SkiaSharp:** Native rendering to a canvas, avoiding WebView overhead but requiring custom C# logic.
Answer Input	**MAUI Entry/Editor** controls for simple numerical and short symbolic answers.	**Custom Math Keyboard Component:** Dedicated MAUI control for complex input (fractions, matrices) to simplify server-side parsing (Advanced Goal).
Back-end API	**Python** using **FastAPI** or **Flask** to expose RESTful endpoints. Chosen for seamless integration with Python ML libraries.	**.NET Core Web API** (C#): For full technology stack uniformity. **Node.js (Express/Koa):** Fast, scalable option using JavaScript/TypeScript.
Communication Protocol	**RESTful API** via **HttpClient** (MAUI) exchanging **JSON** messages.	**gRPC:** High-performance, low-latency communication protocol, but involves higher setup complexity. **SignalR/WebSockets:** For real-time features (overkill for MVP).

1.3.2. Dataset Layer: Content and Quality Assurance

Tabla 1.2: Implementation and Alternatives for the Dataset Layer

Component	Chosen Implementation	Alternative Implementations
Dataset Content	**Bespoke Parametric Templates** (300–600 seeds) aligned with **Spanish Bachillerato** curriculum, ensuring local relevance.	**Direct LLM Generation:** Prompting **DeepSeek-R1** or **GPT-4o** for new problem sets. Requires heavy human filtering for accuracy and style.

Continued on next page

Tabla 1.2: Implementation and Alternatives for the Dataset Layer (Continued)

Component	Chosen Implementation	Alternative Implementations
Problem Generation	Custom **Python Generator** script that substitutes parameter ranges into templates and computes the symbolic solution.	**Fine-tuned Open-Source LLM:** Fine-tune **Mathstral** or distilled **DeepSeek-R1** (using **CoinMath** principles) to translate/rewrite existing English datasets (e.g., MATH) into Spanish parametric templates.
Solution Generation	**Code-Based Rationales** (CoinMath principle): Solutions generated as Python-executable code with descriptive names, promoting verifiability.	**Pure Natural Language LLM:** Faster generation but prone to arithmetic and logical errors, requiring extensive human post-editing.
Validation/QA	**Manual Review** + **Custom Python Verifier** to check if the generated answer from substituted parameters matches the expected symbolic solution.	**MARIO Eval Toolkit Integration:** Hybrid approach (Python CAS + optional LLM checker) for robust, standardized, automated validation of correctness.
Data Storage	**Document Database** (e.g., **MongoDB** or **PostgreSQL** with JSON support) for flexible storage of problem documents including steps and metadata.	**Relational Database** (PostgreSQL/MySQL): Efficient for structured data logging, but less flexible for storing nested solution content.

1.3.3. Recommendation System Layer: Personalization

Tabla 1.3: Implementation and Alternatives for the Recommendation System Layer

Component	Chosen Implementation	Alternative Implementations
Core Algorithm (MVP)	**Bayesian Knowledge Tracing (BKT)**: Interpretable and robust latent Markov process estimating mastery probability (P($Mastery$)) for each skill/tag.	**Heuristics + Spaced Repetition:** Simpler Elo/IRT models combined with fixed half-life regression to schedule review. Suitable for very low-data cold start.

Continued on next page

Tabla 1.3: Implementation and Alternatives for the Recommendation System Layer (Continued)

Component	Chosen Implementation	Alternative Implementations
Problem Selection	**Maximizing Expected Learning Gain:** Selecting problems where P($Mastery$) is in the "Zone of Proximal Development"(e.g., 60-80 % success rate).	**Multi-Armed Bandit (MAB) Algorithms:** UCB or ϵ-greedy to balance the trade-off between exploration (new skills) and exploitation (drilling weaknesses).
Data Logging	**Python Back-end Logger** records detailed interaction logs: *student_id*, *problem_id*, *is_correct*, *timestamp*, *attempts*, and *hints_used*.	**Event Streaming Service** (e.g., **Kafka**): For high-volume, real-time data ingestion, decoupling the logging pipeline from the main API process.
Advanced Algorithm	**Deep Knowledge Tracing (DKT):** Uses RNNs (LSTM) or Self-Attentive models (SAKT) to predict future correctness based on interaction sequence. Requires large data.	**Pre-train DKT on Public Data:** Utilize publicly available MOOC interaction datasets and then fine-tune the model with the app's collected logs to mitigate the cold start problem.
Recommender Implementation	**Python Logic** (simple classes/functions) for BKT update equations integrated directly into the Flask/FastAPI back-end.	**Dedicated C++ Library Wrapper:** Using a high-performance C++ library for the knowledge tracing core wrapped for Python access, if latency becomes an issue.

Capítulo 2

State of the Art

En el estado de la cuestión es donde aparecen gran parte de las referencias bibliográficas del trabajo. Una de las formas más cómodas de gestionar la bibliografía en LaTeX es utilizando **bibtex**. Las entradas bibliográficas deben estar en un fichero con extensión *.bib* (con esta plantilla se proporciona el fichero biblio.bib, donde están las entradas referenciadas más abajo). Cada entrada bibliográfica tiene una clave que permite referenciarla desde cualquier parte del texto con los siguiente comandos:

- Referencia bibliografica con cite: Bautista et al. (1998)
- Referencia bibliográfica con citep: (Oetiker et al., 1996)
- Referencia bibliográfica con citet: Krishnan (2003)

Es posible citar más de una fuente, como por ejemplo (Mittelbach et al., 2004; Lamport, 1994; Knuth, 1986)

Después, LaTeXse ocupa de rellenar la sección de bibliografía con las entradas **que hayan sido citadas** (es decir, no con todas las entradas que hay en el .bib, sino sólo con aquellas que se hayan citado en alguna parte del texto).

Bibtex es un programa separado de latex, pdflatex o cualquier otra cosa que se use para compilar los .tex, de manera que para que se rellene correctamente la sección de bibliografía es necesario compilar primero el trabajo (a veces es necesario compilarlo dos veces), compilar después con bibtex, y volver a compilar otra vez el trabajo (de nuevo, puede ser necesario compilarlo dos veces).

2.1. Existing Datasets

Many math datasets have been created for different tasks. Our problem requires a data set with very accurate labels and a comprehensive answer in steps that may clear the students doubts.

2.1.1. Math dataset

The Math dataset has 12500 problems taken from math competitions. The architecture of the database provides for each problem the problem statement, the tag, the level, and the explained answer. It was used to test LLMs performance in 2021 when they scored between 3 % to 6.9 % of correct answers, while computer science PhD students scored 40 %. These data may seem harsh against the models tested, however, new data show that nowadays models score higher than 40 %. This dataset has proven to have several usages, such as training LLMs. But if it were to be used for helping students who may be struggling at school, the high level of difficulty from the problems might make the issue worse rather than help the student.

2.1.2. GSM8K

GSM8K is a dataset with 8.5K math problems that provide the statement and a natural language answer. It focuses specifically on multi-step mathematical reasoning in the context of grade school word problems and it has been used to improve LLMs performance. The complexity is moderate, requiring between 2 to 8 steps to solve each problem. It used to be difficult for LLMs, DeepSeek-Math-7B-RL model achieved 58.8 % of accuracy, while DeepSeek-R1-Distill-Llama-8B achieved 89.1 %. In order to train LLMs to solve the dataset they used two methods: As a baseline, finetuning a generator model on the training set. Then, by training verifiers, used to rank the correctness of each solution, it can simulate the multi-step reasoning.

2.1.3. AMPS dataset

The AMPS dataset was introduced in 2021 alongside MATH with millions problems providing step-by-step solution with the purpose of training LLMs to understand basic math principles. It is divided in two different parts: The Khan Academy Component: With aproximately 100,000 problems that cover the standard K-12 math curriculum. The Mathematica Component: Has over 5 million generated problems problems using Mathematica software.

2.1.4. Orca-Math

Orca-Math is a synthetic dataset of 200,000 problems that was produced using other datasets like GSM8K and LLMs to produce new problems from the existing ones and write a a full answer providing how to get there. The dataset gives us only the problem statement and an the step by step explanation of how to solve the problem with the answer written on it. The objective of creating this dataset was to train Small Language Models (SMLs) to solve math problems at a higher level. A Mistral-7B model was trained on this dataset and achieved 86.81 % accuracy on GSM8k, which was significantly better than Llama-2-70B, Gemini Pro and GPT-3.5.

Benchmark	Dataset	DeepSeek-Math-7B-RL	DeepSeek-R1-Distill-L
MATH	Math Competition (Hard)	**58.8 %**	**89.1 %**
GSM8K	Grade School Math	**88.2 %**	**89.6 %**
AIME 2024	Elite Math Olympiad	**∼10–15 %** (est.)	**72.6 %**

2.2. Large Language Models

In order to make the best out of these datasets, we might need to transform their structure in order to fit the application format. We may also need the steps to reach the solution. One solution to these issues is to use LLMs as long as the solutions they provide are accurate.

2.2.1. DeepSeek-R1

The architecture used is Mixture-of-Experts(MoE). It has a massive total parameter count ($\sim$ 671B), but only a small fraction ($\sim$ 37B) is activated per token, making inference significantly faster than a fully dense model of that size. It has a dedicated Reasoning Model that prioritizes detailed, step-by-step logic over fast, direct answers, which will prove to be very effective when solving mathematical problems. The Context Window is very large 128,000 tokens, allowing it to handle complex, multi-page problems or vast contexts for advanced reasoning. The model weights are released under the highly permissive MIT License, allowing us to use it. Performance of different models with different datasets:

2.2.2. Mathstral

Mathstral is a model built on Mistral 7B transformer architecture, specialized on mathematical reasoning. It supports 32,000 of tokens context window. The model is very efficient and can be used with a less powerful GPU, since it has 7B parameters. It is Open-Source for the weights under Apache 2.0 License.

There exist other alternatives, such as Wolfram Alpha, which could be a good tool for manual verifications through the web, as using the API requires payment. These models can also be used as a chatbot to answer the user's questions on a specific problem.

2.2.3. Other studies and tools on the issue

2.2.3.1. MARIO Eval

MARIO Eval is a toolkit design to leverage numerical accuracy and natural language comprehension in mathematical datasets. It solves the lack of generalizability in math datasets, since each datasets design its own evaluation scipt, and it prevent inconsistencies. It uses a hybrid approach built with a Python Computer Algebra System(CAS) for numerical precision and an optional LLM for the evaluation. This toolkit achieves an improved evaluation for math datasets.

2.2.3.2. CoinMath

CoinMath studies different ways of improving LLM's math performance. One way is to use concise comments, which helps the model align the code structure with the natural language reasoning steps. Another way is to use descriptive naming for variables and functions, which shows an enhancement on reasoning. Finally, including hard-coded solutions improves performance. These techniques showed a significant improvement from the baseline model, MAmmoTH, which was the State-of-the-Art Math LLM used.

2.3. Other math tutors

2.3.1. Duolingo

The Duolingo application uses a combination of diagnistic engines, business rules, heuristic models for selecting items and a lot of product engineering as they describe in their Duoling Papers. 1. The Duolingo learning method consists in: Learn by doing: You don't need an introduction to be able to do the problems. They learn through repetition and pattern acquisition, formally defined in the field of statistical learning. 2. Learn in a Personalized Way: Using machine learning-based models (Birdbrain), using millions of problems solved daily, understands how prolific the student is and how difficult the problems are. The model will focus on the range of difficulty appropriate for the student's learning. 3. Focus on What Matters: The courses are designed based on the Common European Framework of Reference (CEFR). 4. Stay Motivated: The hardest part of learning is staying consistent. Duolingo's solution is to use gamification techniques like Win-Streaks. 5. Feel the delight: They use characters and stories that help make learning more fun. Additionally, the use of various visual tools for problem-solving helps make learning more entertaining. Duolingo uses a system called Birdbrain that uses several architectures. Its main model is based on a Long Short-Term Memory (LSTM), which predicts student performance based on their knowledge status, while also being able to analyze sequential data over time effectively. It also uses a statistical model to choose when to review previously viewed material. Moreover, it uses a different model that uses half-life regression, which identifies concepts that are often forgotten over time using the error patterns of millions of users. An other model that is used is an adaptive review scheduling to choose where and when to insert these problems to maximize user retention. Finally, Duolingo uses a bandit algorithm for its notification system.

2.3.2. Khan academy

Khan academy is a non-profit that offers a massive library of courses and it has a Course Mastery feature which works as a recommendation system, which works by giving the user a Course Challenge that evaluates the ability of the student and adjust the recommendation accordingly.

It also integrates a generative AI tutor called Khanmigo, that works by helping students get to the answer rather than giving it right away.

2.3.3. Brilliant.org

Brilliant is a math and science learning platform that works with interactive problems. It uses an adaptive learning system that provides personalized recommendations to the users.

2.3.4. Prodigy Math

A platform focused on students that have 1-8th grade math, but the gamification of this app outlines very well the core of adaptive learning, using an AI that starts with an initial diagnostic for a student and adjust as the student progresses.

2.3.5. Application technologies

2.4. Existing recommendation systems

2.5. Recommendation system technologies

2.5.1. Knowledge tracing

Given a set of interactions of a student's outcomes x0..xt, having for each interaction a tuple with the question tag and whether the student's answer was correct or incorrect: xt = qt, at. What we want to do is predict aspects of the next interaction xt+1, such as whether the answer would be correct depending on the type of question asked. This task is called Knowledge tracing and there are several approaches for it.

2.5.2. BKT

Bayesian Knowledge Tracing is the most common approach to student learning models. Knowledge is transformed into binary variables, using a Hidden Markov model. Model (HMM) probabilities are updated. There are techniques to refine these concepts, such as Cognitive Task Analysis, which uses path thinking when solving problems to broaden the scope of information in the binary response.

2.5.3. LSTM

Long Short-Term Memory is a type of Recurrent Neural Network (RNN) used to handle long-term dependencies. It uses three types of gates: Forget Gate (decides what information to discard from each Cell State), Input Gate (decides what part of the new information to store in the Cell State), and Output Gate (chooses what part of the Cell State to move to the Hidden State). The Cell State represents the main memory stream, and the Hidden State represents the short-term memory stream.

2.5.4. DKT

Applies a combination of two types of RRNs: a vanilla RRN with sigmoid units and a LSTM model. The connection of variables in an RNN goes from X (input) through H (hidden state) and ends in Y (output).

(Image)

Equations, where $(\cdot)$ is the sigmoid function:

(Figure)

Equations for LSTM:

(figure)

To convert these interaction into input vectors of fixed length we have two methods depending on the number of M problems:

- Low M: Set xt to be one-hot encoding of tuple ht={qt, at}. $xt \in \{0,1\}^{2M}$.
- High M: Assign a random vector nq,a N (0, I) to each input tuple and set each x to the random vector xt = n(qt,at).

The output yt is a vector of length = M (number of problems) and represents the predicted probability of the student answering correctly the problem. Therefore, the prediction a(t+1) can be obtained from the entry in yt corresponding to q(t+1).

The training objective is the negative log likelihood of the students responses. The loss for a single student is defined by this function:

(figure)

(qt+1) will be the one-hot encoding from the exercise answered in t+1 and l() is the binary cross entropy, l(y. . .) is the loss of one prediction.

The objective is minimized using stochastic gradient descent on minibatches. A dropout is applied to ht when computing yt and not when computing the next hidden state to avoid overfitting. The models used a hidden dimensionality of 200 and a mini-batch size of 100.

To understand the relationship between the problems an influence function (Jij) is defined for each pair of exercises and a synthetic dataset will be created so that the model understand the structure of skills or problem types.

(figure)

The results seen in the DKT study show an AUC of 0.68 for the BKT on the Khan dataset, while the LSTM neural network model led to an AUC of 0.85.

(figure)

The results regarding the structure of the problems can be visualized in a graph of conditional influences where a directed show if performance on an exercise A changes, how much does that influence the predicted performance on exercise B (A->B).

Applying this method on Khan academy dataset produces the following graph, only using pairs (A,B) of exercises if B appears more than 1 % of times after A:

(figure)

The conclusion is to make an implementation with hybrid architecture: heuristic rules + probabilistic model (BKT).

Capítulo 3

Project Description

3.1. Dataset layer: collecting and generating problems.

The strategy here is to select a dataset as good as possible to use as basis. Expectedly , not a single one out of the publicly available ones is good enough for one reason or another, they may lack relevant features such as tags or topics.

Due to this, after selecting an appropriate dataset as our base, the next step is to build from it a better dataset. We will cover how to achieve this after.

3.1.1. Base dataset

I have decided to use GSM8K as the starting dataset because it is the one that balances these important principles for a dataset of this kind.

- **Quantity:** An app that tries to give a personalized experience needs to have enough items to choose from. Therefore, it is strictly necessary to achieve a sufficient amount of problems from each topic.

- **Structure:** For the sake of the app itself, the dataset will need to provide the problem statement and answer in good form.

3.1.2. How to transform the dataset?

With the method of construction that we are using for our final dataset of problems, we will essentially only need the problem statement, since the rest will be automatically generated.

This is due to the fact that we are using Mathstral, which is a Large Language Model, to read from GSM8K and generate the additional information needed for our platform. This process was divided into three phases so the execution could be more efficient, since each phase required a different prompt and is asked to write different amounts of information. The phases in order of execution are:

- **Phase A: tagging + course + difficulty only:** This phase receives the defined taxonomy and the dictionary with Knowledge Components (KC) and tags available to use.
- **Phase B: statement normalization + final answer:** This phase normalizes the problem statement and final answer, translating them into Spanish.
- **Validation phase:** During this phase, problems will be reviewed to check if the tags fit the problem, so if they do not they will be marked as invalidated and in a future iteration it is possible to correct them.
- **Phase C: solution steps:** After the validation phase succeeds, this phase will generate logical steps to get to the desired solution, available gradually for users in the application.

3.2. Recommendation system layer.

There are plenty of different approaches to recommendation systems. The well known ones that I have studied use complex structures such as multiple Neural Networks or DKT.

For this project I have decided to use a hybrid model that combines a Heuristic model and a Bayesian Network based around the BKT model. The BKT estimates knowledge and the heuristics decide what to do with that knowledge.

The main objective of a good recommendation system should be balancing these four metrics:

- **Preference:** The user receives problems from the tags and difficulty he wants.
- **Need:** The system tries to fix recurrent mistakes or weak topics.
- **Exploration:** The system should try to acquire recent performance data of all topics.
- **Session shape:** Each session should be coherent and have a natural flow.

3.2.1. Conceptual architecture

Essentially, the system is solving: *Given a student with a partial known knowledge across different KCs, what is the next optimal problem to maximize learning?*

The system decomposes into:

1. **State estimation (BKT):** P(Lkc) = probability of the student knowing a Knowledge Component.
2. **Heuristic policy (scoring):** score(kc) = f(interest, need, uncertainty, diversity)
3. **Sampling policy (softmax):** The probability P(kc) is proportional to $e^{\log(score)/T}$.

Parameter	Symbol	Meaning (Probabiliy of)
Initial knowledge	P(Lo)	Student knows the KC already
Learning rate	P(T)	Learning KC after one attempt
Slip	P(S)	Answer incorrectly despite knowing
Guess	P(G)	Answer correctly without knowing

Tabla 3.1: BKT parameters

4. **Shaping constraints:**
    - Number of problems.
    - Repetition/diversity penalties.
    - Difficulty controller

3.2.2. BKT Model

A Bayesian Knowledge Tracing model is able to predict somewhat accurately whether the user is actually learning or not. It is capable of selecting the recommended problems in such a way that the knowledge acquired is optimal.

In reality, BKT is a Hidden Markov Model (HMM) in disguise. For our problem, each KC is modeled as a 2-state HMM, where:

- L=0 means do not know.
- L=1 means knows.

This table shows the BKT variable parameters for each KC:

The value that we are mostly interested in is P(L), since this gives us whether the user knows a KC or not.

After one attempt, the parameters will update using these formulas: **Evidence update:** $P(L \mid correct)$ = P(L)*(1-P(S)) / (P(L)*(1-P(S))+(1-P(L))*P(G)) $P(L \mid incorrect)$ = P(L)*P(S) / (P(L)*P(S)+(1-P(L))*(1-P(G)))

Learning update: (even if they failed, they might have learned) P(Ln+1) = P(L|evidence) + (1-P(L|evidence))*P(T)

At the beginning, the system uses a cold start trick that consists on mapping the user preferences as prior knowledge. In other words, since we do not know how much a user knows at the start, we use the scoring for each kc that they provided.

3.2.3. Heuristic Model

In order to get an idea of what the user wants in terms of problem, the user will be able to do a questionnaire that establishes the initial weights for some aspects of the recommendation system. The questionnaire gets values for:

- Scores for each Knowledge Component (KC) or main topic from 1-5.
- Target difficulty.

- Cross-course policy: allow to recommend problems from above or below courses depending on performance, encouraging forward courses more than lower courses.
- Objective-dependent behavior.

3.2.3.1. Scoring

Initially, The scoring system for each KC is obtained from this formula: Score = wI*Ikc + wN*Nkc + wU*Ukc + wD*Dkc - Precent - Prepeat Where:

- Ikc: normalized questionnaire interest from user preferences.
- Nkc: learning need from BKT when available, else from accuracy.
  - Without BKT: N = 1 - accuracy
  - With BKT: N = 1 - P(Lkc)
- Ukc: uncertainty: high when attempts are few. U = 1 / (1 + attempt/pi)
- Dkc: diversity bonus: higher if a KC is not in the current session. D = 1 / (1 + session_count)
- Wi... Wd: weights determined by the user's primary objective.
- Precent and Prepeat: penalties

3.2.3.2. Softmax Sampling

A KC is selected using: P(KC) prop exp(log(score)/T), where T = temperature

I have chosen to use softmax in order to introduce controlled randomness that does not overfit the top KC. A low value for T will be more greedy, while a high value will be more exploratory. Currently we use 0.35 to stay focused on the recommended kc from the scoring system. It is important that we have some level of randomness to the recommendation system, so that two identical students may experience different learning paths.

3.2.3.3. Session-Level Dynamics

When generating each session, the backend is tracking each KC, tag, difficulty and course repetition counts. With this information, the system computes a penalty for recent history in different session and a penalty within the session.

3.2.3.4. Objective-dependent behavior

The **objective-dependent behavior** is classified in four different archetypes:

- **PRACTICE**: will keep balanced weights

accuracy	speed	difficulty*
low	-	decrease
medium	-	keep
high	fast	increase
high	ok	increase slightly
high	slow	keep
low	slow	decrease strongly

Tabla 3.2: Fuzzy rules

- **IMPROVE_GRADES**: prioritizes weak KCs and KCs from their interests, allowing more repetition.
- **REVIEW**: seeks more diversity across the topics with less exploration.
- **LEARN_NEW_CONTENT**: boosts unseen KCs and allows more forward difficulty or course movement.

3.2.3.5. Cross-course policy

If the **cross-course policy** is true, it will select the course based on:

- With weak performance will selects current and with a low chance previous course.
- With medium performance will mostly select the current course.
- Strong performance will select current and next course.

3.2.3.6. Difficulty control

For selecting the difficulty we represent the user performance in two variables: **accuracy** and **speed**. Using these variables we create a set of rules that represent the desired logic.

The formula looks like this: d* = dprofile + fuzzy Where:

- dprofile is the target difficulty that the user selected in the questionnaire.
- fuzzy value depends on the accuracy and speed.

The variables can take these different values:

- **accuracy**: low, medium or high. Triangular functions.
- **speed**: slow, ok or fast. Trapezoidal functions.

These are the rules, where the output variable is difficulty*: The fuzzy output is a continuous shift between -1.5 and 1.5 that gets added to the target difficulty.

This approach produces something close to human tutoring behavior, trying to push the student to learn more when they are performing well or when they are performing worse. We are treating this problem as if it was a PID-kind of problem, bringing in control systems as a solution.

3.2.4. Takeaways

The main aspect of the real learning process to take into account when making this rules is to understand where is the sweet spot when real learning is happening. If the user completes everything fast and without mistakes he is most likely not learning anything new, that is when we can increase the difficulty without a doubt. However, for the rest of situations we want to take into account the possibility of the user having very different performances depending on the level of energy, mood or other things out of our reach, so we should not change extremely the difficulty in these cases.

The main thing to promote here is a bit of a challenge, without expecting too much from the user nor too little.

The main limitation for BKT is that we consider knowledge to be timeless, while after a period of time it is possible that a student has forgotten something they already knew. Also, it treats knowledge as a binary variable, you either know something or you do not know it. The forgetting aspect is slightly prevented by uncertainty (U) heuristic, since it tries to get as many attempts of each KC as possible, and it is proportional, so no KC gets behind after some time.

Moreover, the system treat each KC as independent pieces of knowledge, although in reality these topcis have dependencies between each other.

3.3. Application layer.

The architecture of the application uses a MAUI cross-platform client, a FastAPI service and MongoDB for the database. The client and server communicate over HTTP using JSON, ensuring that the client has no access to the database and the server does not interact with the UI. This separation is key for the same backend to serve a Windows desktop, Android and iOS app. It is also extremely convenient for the implementation aspect, as modifying code from the back-end or front-end will not break the other side as long as the set of endpoints are respected.

3.3.1. Backend

The backend is implemented in python. The main reason being that we will be using Ollama to use LLM calls locally. This is used for the dataset generation and for the chatbot itself in the running app. Therefore, for simplicity, I decided to program the whole backend in python.

The backend application is implemented using FastAPI library, which allow to process the different HTTP request we define in isolation. The server works with the context that provides each request from its path or its body independently from other requests, applying the standard rest property called statelessness. This allows the user state to be safe when restarting the server, but requires all useful state to be stored in the MongoDB database.

3.3.1.1. Routes

The endpoints are divided into four groups, that are specified in different routers of the FastAPI application. The four routers are:

- **users**: owns the user profile, interest and BKT state.
- **problems**: owns the problems lookup.
- **sessions**: owns the recommendation and the attempts results.
- **tutor**: owns the chatbot assistance for solving problems.

Every endpoint declares its request and response bodies with Pydantic classes that are creating using the BaseModel class from FastAPI.

Each endpoint allows for different verb semantics that hold different properties and are implemented in FastAPI to be used as python decorators on our router functions. These are the ones that I used:

- **GET**: readonly action, it is used for the backed to fetch problems.
- **POST**: mutates the state and is not idempotent – calling /session/start twice will generate two sessions – , it is used for sending an attempt during a session and for receiving the chatbot answer.
- **PUT**: updates one resource and is idempotent – calling PUT /users/{id}/iterests multiple times with the same body does not change the system after the first one –, it is used for updating the user interests after they redo the questionnaire.

The main benefits of this approach are automatic validation – when there is a missing field or a wrong type the system returns 422 error message – and automatic documentation that can be accesed in /docs page.

3.3.1.2. Scripts

Python scripts structure:

- app/
- routes/
  - problems.py ->router that defines http endpoints regarding the problems.
  - sessions.py ->router that defines http endpoints regarding the user session (daily problems, results in problems, updating bkt).
  - tutor.py ->router that defines http endpoints to send chatbot response.
- main.py ->main entry point for fastapi backend, creates backend application.

- ml/
  - bkt/
    - bkt_runtime.py ->implements functions to get the user bkt state, update the bkt state of a user and recommend a set of problems to a user.
    - heuristic.py ->implements a heuristic recommendation model using fixed weights and the user interests to select n problems.
  - chatbot/
    - chatbot.py ->implements the methods that call the chatbot model (deepseek-r1) using ollama and generate the answer to a question of a specific problem.
  - dataset/
    - transformer.py ->implements the generation of the dataset methods during 3 phases that can be executed individually, loads the existing gsm8k dataset and uses KC/tags and taxonomy for consistency.
- scripts/
  - build_dataset.py ->implements main function to generate the dataset using transformer.py modules.

3.3.2. Frontend

The **MVVM pattern** is appropriate because MAUI provides a built-in binding system and dependency injection. The framework has embraced modern patterns, including MVVM and MVU, and the **.NET Community Toolkit** reduces boilerplate code for view modelsappisto.app. MVVM will allow a clean separation of views (pages), view models (state and commands) and models (data), simplifying unit testing and facilitating data binding to our recommendation engine.

4 main pages:

Daily lesson page - presents a curated set of 5-10 problems for the day. The user taps a "Start" button, sees each problem one at a time, submits an answer and receives feedback.

Problem library page - displays all available problems organised by **topic**, **course** and **difficulty level**. A list or grid can show the problem title and tags; tapping an item opens a dedicated problem page.

Profile/interests page - allows students to select their interests (e.g. calculus, probability) from the questionnaire which seed the recommender and show statistics on completed problems.

Settings pages - provides different options to modify the configuration of the application, such as turing dark mode on or off.

2 secondary pages:

Problem Detail page - opens the chosen problem from the current session or the problems library, allowing to solve it, ask for hints or ask a question to the chatbot.

Questionnaire page - opens from the profile page and makes the questionnaire to the user, asking him to rate each Knowledge Component, the course he wants to study, the difficulty and his learning objective.

Class structure in .net maui:

- Models/
  - ProblemOut.cs
  - ProblemDetailOut.cs
  - StartSessionRequest.cs
  - StartSessionResponse.cs
  - AttemptRequest.cs
  - AttemptResponse.cs
  - TutorChatRequest.cs
  - TutorChatResponse.cs
  - UserInterestRequest.cs
  - UserInterestResponse.cs
- Services/
  - ProblemService.cs
  - SessionService.cs
  - TutorService.cs
  - UserService.cs
- Pages/
  - MainPage.xaml
  - ProblemsLibraryPage.xaml
  - ProblemDetailPage.xaml
  - ProfilePage.xaml

    - QuestionnairePage.xaml
    - SettingsPage.xaml
- PageModels/
    - MainPageModel.cs
    - ProblemsLibraryPageModel.cs
    - PriblemDetailPageModel.cs
    - ProfilePageModel.cs
    - QuestionnairePageModel.cs
    - SettingsPageModel.cs
- ApiService.cs
- App.xaml
- AppShell.xaml
- GlobalUsings.cs
- MauiProgram.cs

Here are some screenshots of the most relevant pages and interactions of the application for windows desktop. 3.1

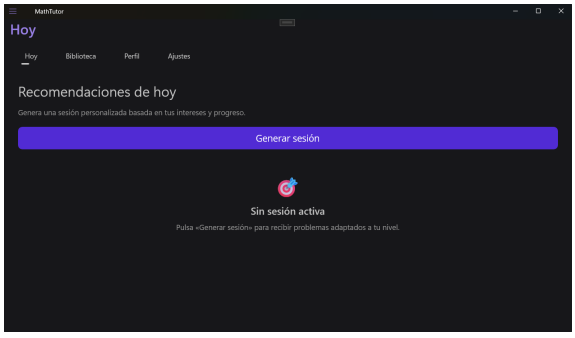

Figura 3.1: Main Page empty

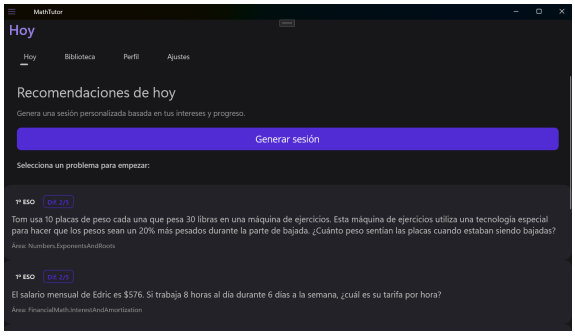

Figura 3.2: Main Page full

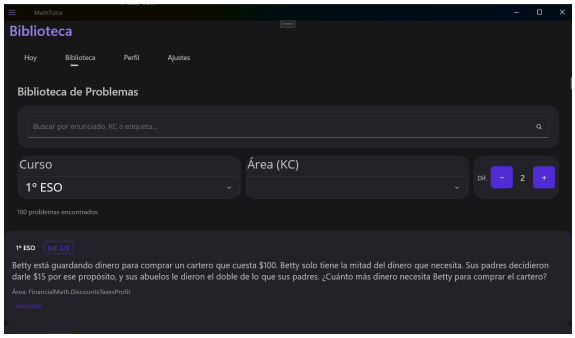

Figura 3.3: Problems Library Page

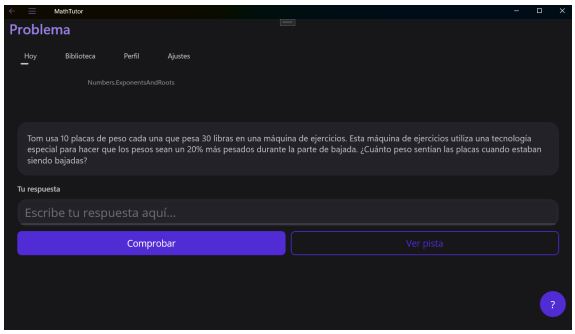

Figura 3.4: Problem Detail Page regular

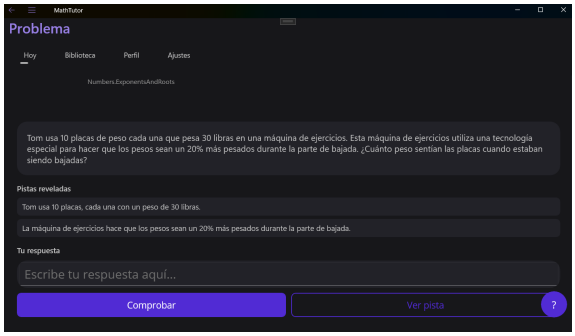

Figura 3.5: Problem Detail Page hints

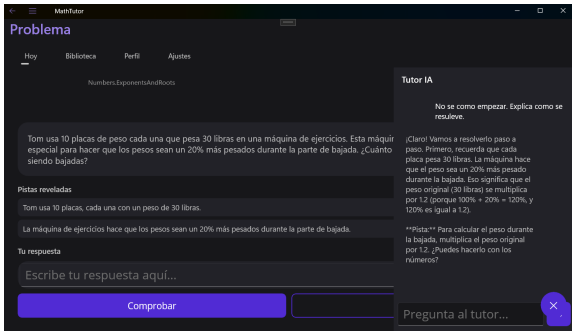

Figura 3.6: Problem Detail Page chatbot

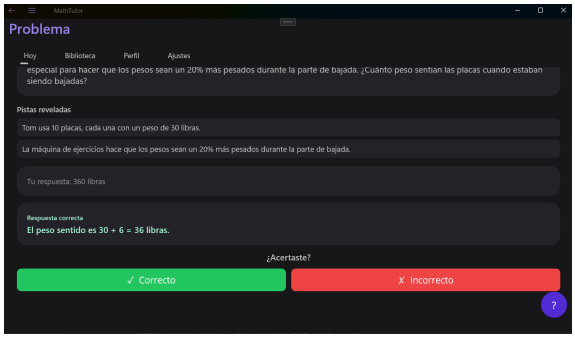

Figura 3.7: Problem Detail Page answer

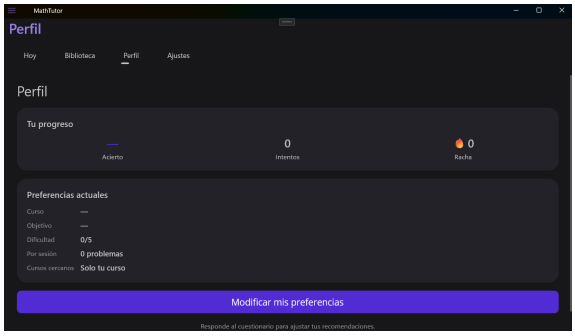

Figura 3.8: Profile Page

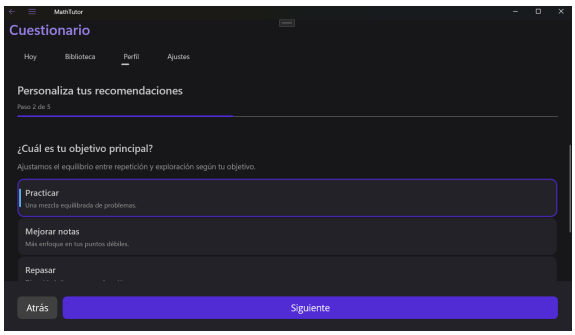

Figura 3.9: Questionnaire Page

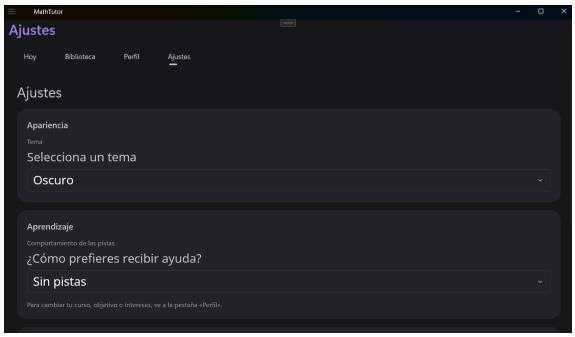

Figura 3.10: Settings Page

Capítulo 4

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 4.

Contribuciones Personales

En caso de trabajos no unipersonales, cada participante indicará en la memoria su contribución al proyecto con una extensión de al menos dos páginas por cada uno de los participantes.

En caso de trabajo unipersonal, elimina esta página en el fichero `TFGTeXiS.tex` (comenta o borra la línea `\include{Capitulos/ContribucionesPersonales}`).

Estudiante 1

Al menos dos páginas con las contribuciones del estudiante 1.

Estudiante 2

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

Y así, del mucho leer y del poco dormir, se le secó el celebro de manera que vino a perder el juicio.
(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

Bautista, T., Oetiker, T., Partl, H., Hyna, I. y Schlegl, E. *Una Descripción de LaTeX 2ε*. Versión electrónica, 1998.

Knuth, D. E. *The TeX book*. Addison-Wesley Professional., 1986.

Krishnan, E., editor. *LaTeX Tutorials. A primer*. Indian TeX Users Group, 2003.

Lamport, L. *LaTeX: A Document Preparation System, 2nd Edition*. Addison-Wesley Professional, 1994.

Mittelbach, F., Goossens, M., Braams, J., Carlisle, D. y Rowley, C. *The LaTeX Companion*. Addison-Wesley Professional, segunda edición, 2004.

Oetiker, T., Partl, H., Hyna, I. y Schlegl, E. *The Not So Short Introduction to LaTeX 2ε*. Versión electrónica, 1996.

Apéndice A

Título del Apéndice A

Los apéndices son secciones al final del documento en las que se agrega texto con el objetivo de ampliar los contenidos del documento principal.

Apéndice B

Título del Apéndice B

Se pueden añadir los apéndices que se consideren oportunos.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.

Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes

–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.

Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes